

UNIT I

DATA MODELS AND QUERYING

A **database-management system (DBMS)** is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the **database**, contains information relevant to an enterprise.

The primary goal of a DBMS is to provide a way to store and retrieve database information that is both *convenient* and *efficient*.

Database-System Applications.

- **Banking**-For customer information, account activities, payments, deposits, loans etc.
- **Airlines**-For reservations and schedule information.
- **Universities**- For student information, course registrations, colleges and grades.
- **Telecommunications**-It helps to keep call records, monthly bills maintaining balances etc.
- **Finance**: For storing information about stock, sales and purchases of financial instruments like stocks and bonds.
- **Accounting**: Database systems are used in maintaining information employees, salaries and payroll taxes
- **Manufacturing**: For management of supply chain and tracking production of items in factories database systems are maintained.
- **Reservations Systems**: In airline/railway reservation systems, the database is used to maintain the reservation and schedule information.

1.1 PURPOSE OF DATABASE SYSTEMS

To allow users to manipulate the information, the system has a number of application programs that manipulate the files, including programs to:

- Add new records
- Update the content in the records
- Delete the records

File Processing System

- This typical **file-processing system** is supported by a conventional operating system.

- The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to, the appropriate files. Before database management systems (DBMSs) were introduced, organizations usually stored information in such systems.
- Keeping organizational information in a file-processing system has a number of major **disadvantages**:
 - ✓ Data redundancy and inconsistency
 - ✓ Difficulty in accessing data
 - ✓ Data isolation
 - ✓ Integrity problems
 - ✓ Atomicity problems
 - ✓ Concurrent-access anomalies
 - ✓ Security problems

Data redundancy and inconsistency

This redundancy leads to higher storage and access cost. In addition, it may lead to **data inconsistency**; that is, the various copies of the same data may no longer agree.

Difficulty in accessing data

Conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner. More responsive data-retrieval systems are required for general use.

Data isolation

Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.

Integrity problems

The data values stored in the database must satisfy certain types of **consistency constraints**.

Atomicity problems

A computer system, like any other device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be restored to the consistent state that existed prior to the failure.

Concurrent-access anomalies

For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously.

Security problems

Not every user of the database system should be able to access all the data. For example, in a university, payroll personnel need to see only that part of the database that has financial information.

Advantages of DBMS:

- Redundancy can be reduced
- Inconsistency can be avoided
- The data can be shared
- Standards can be enforced
- Security can be enforced
- Integrity can be maintained

Disadvantages of DBMS:

- Confidentiality, Privacy and Security
- Enterprise Vulnerability
- The cost of using a DBMS

Database Systems	Conventional File systems
Data redundancy is less	Data redundancy is more
Security is high	Security is very low
Database systems are used when security constraints are high	Conventional file systems are used where there is less demand for security constraints.
Database systems define the data in a structured manner. Also there is well defined co-relation among the data.	File systems define the data in un-structured manner. Data is usually in isolated form.
Data inconsistency is less in database systems.	Data inconsistency is more in database systems.

1.2 VIEW OF DATA

A major purpose of a database system is to provide users with an *abstract* view of the data.

- **Data Abstraction**
- **Instances and Schemas**
- **Data Models**

Data Abstraction

Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

- **Physical level**

The lowest level of abstraction describes *how* the data are actually stored. The physical level describes complex low-level data structures in detail.

- **Logical level**

The next-higher level of abstraction describes *what* data are stored in the database, and what relationships exist among those data.

Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

- **View level**

The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database.

Figure 1.1a shows the relationship among the three levels of abstraction.

Many high-level programming languages support the notion of a structured type. For example, we may describe a record as follows:

```
type instructor = record  
  ID : char (5);  
  name : char (20);  
  dept name : char (20);  
  salary : numeric (8,2);  
end;
```

This code defines a new record type called *instructor* with four fields. Each field has a name and a type associated with it. A university organization may have several such record types, including

- *department*, with fields *dept name*, *building*, and *budget*
- *course*, with fields *course id*, *title*, *dept name*, and *credits*
- *student*, with fields *ID*, *name*, *dept name*, and *tot cred*

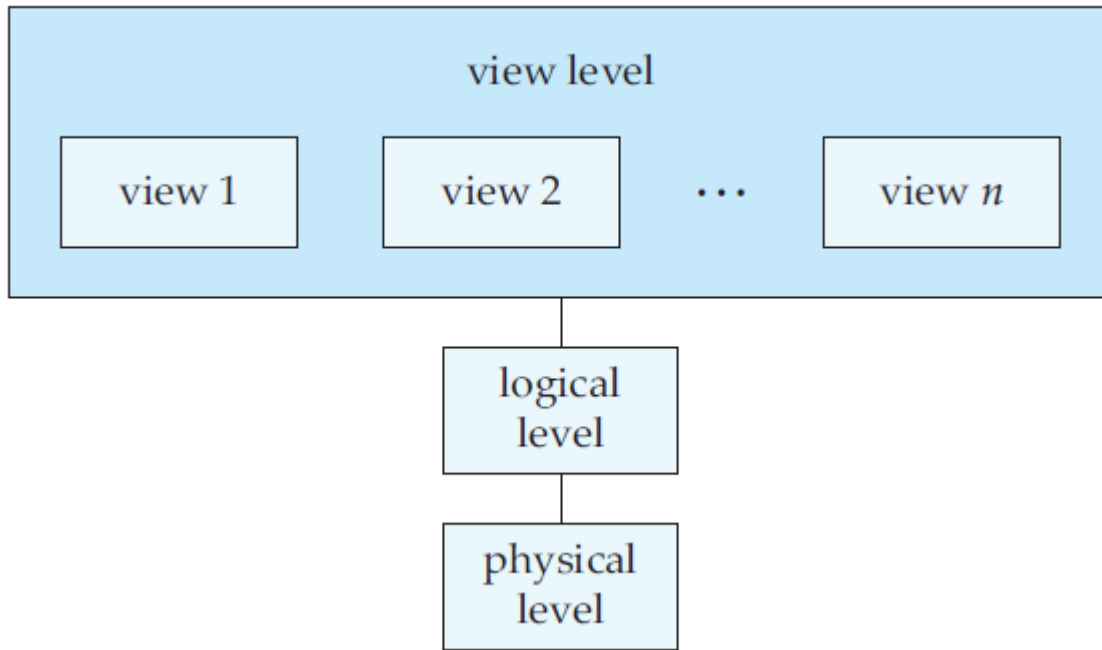


Fig 1.1 a Three levels of data abstraction

Finally, In fig 1.1 at the view level, computer users see a set of application programs that hide details of the data types.

At the view level, several views of the database are defined, and a database user sees some or all of these views.

In addition to hiding details of the logical level of the database, the views also provide a security mechanism to prevent users from accessing certain parts of the database.

1.2.1 Instances and Schemas

The overall design of the database is called the database **schema**.

Example:

Roll No	Name	Marks
---------	------	-------

The collection of information stored in the database at a particular moment is called an **instance** of the database.

Example:

Roll No	Name	Marks
1011	AAA	68
1012	BBB	81

Types of Schema:

Database systems have several schemas, partitioned according to the levels of abstraction.

- The **physical schema** describes the database design at the physical level of abstraction.
- the **logical schema** describes the database design at the logical level of abstraction.
- A database may also have several schemas at the view level, sometimes called **subschemas**, that describe different views of the database.

Database Languages:

A database system provides a **data-definition language** to specify the database schema and a **data-manipulation language** to express database queries and updates.

Data-Manipulation Language

A **data-manipulation language (DML)** is a language that enables users to access or manipulate data as organized by the appropriate data model. The types of access are:

- Retrieval of information stored in the database
- Insertion of new information into the database
- Deletion of information from the database
- Modification of information stored in the database

There are basically two types:

- **Procedural DMLs** require a user to specify *what* data are needed and *how* to get those data.
- **Declarative DMLs** (also referred to as **nonprocedural DMLs**) require a user to specify *what* data are needed *without* specifying how to get those data.

A **query** is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a **query language**. Although technically incorrect, it is common practice to use the terms query language and data-manipulation language synonymously.

Data-Definition Language

- Data Definition Language(DDL) is a specialized language used to specify a database schema by a set of definitions.
- It is a language which is used for creating and modifying the structure of tables, views ,indexes and so on.
- DDL is also used to specify additional properties of data.

- Some of the common commands used in DDL are CREATE,ALTER,DROP.

1.3 DATA MODELS

Underlying the structure of a database is the **data model**: a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.

1.3.1 Different types of data model

- Entity-relationship model(Graphical/Pictorial representation model)
- Relational model(Record based model)
- Object relational model(Programming model)
 - Object Oriented model
 - Object Relational model
- Network model
- Hierarchical model

1.3.1.1 Entity-Relationship Model

The entity-relationship (E-R) data model is based on a perception of a real world that consists of a collection of basic objects, called *entities*, and of *relationships* among these objects(Fig 1.1b)

- An **entity** is a “thing” or “object” in the real world that is distinguishable from other objects.
- A **relationship** is an association among several entities.
- Entities are described in a database by a set of **attributes**.

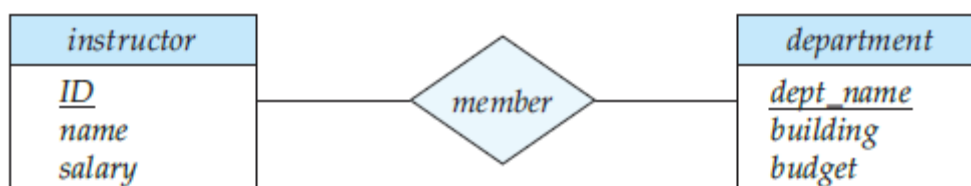


Fig: 1.1 b. A Sample ER Diagram

1.3.1.2 Relational Model

The Relational Model uses a collection of tables both data and the relationship among those data. Each table have multiple columns and each column has a unique name.

Relational database comprising of two tables

Customer –Table.

Customer-Name	Security Number	Address	City	AccountNumber
Preethi	111-222-3456	Yelhanka	Bangalore	A-101
Sharan	111-222-3457	Hebbal	Bangalore	A-125
Preethi	112-123-9878	Jaynagar	Bangalore	A-456
Arun	123-987-9909	MG road	Bangalore	A-987
Preethi	111-222-3456	Yelhanka	Bangalore	A-111
Rocky	222-232-0987	Sanjay Nagar	Bangalore	A-111

Account –Table

Account-Number	Balance
A-101	1000.00
A-125	1200.00
A-456	5000.00
A-987	1234.00
A-111	3000.00

Customer Preethi and Rocky share the same account number A-111

Advantages

1. The main advantage of this model is its ability to represent data in a simplified format.
2. The process of manipulating record is simplified with the use of certain key attributes used to retrieve data.
3. Representation of different types of relationship is possible with this model.

1.3.1.3 Object Relational Model (Programming Model)

1.3.1.3.1 Object-Oriented Data Models

- Several models have been proposed for implementing in a database system.
- One set comprises models of persistent O-O Programming Languages such as C++ (e.g., in OBJECTSTORE or VERSANT), and Smalltalk (e.g., in GEMSTONE).
- Additionally, systems like O2, ORION (at MCC – then ITASCA), and IRIS (at H.P.-used in Open OODB).

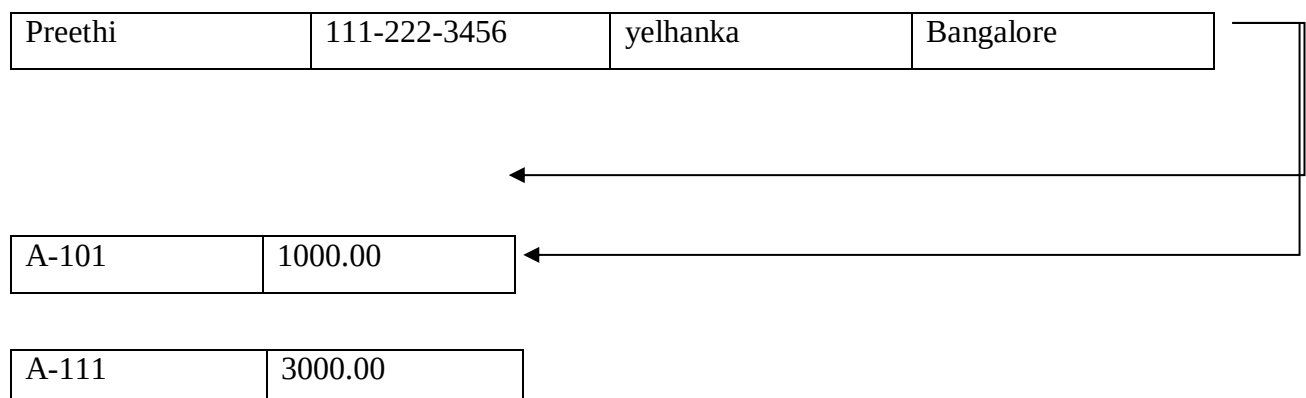
1.3.1.3.2 Object-Relational Models

- Most Recent Trend. Started with Informix

- Universal Server.
- Relational systems incorporate concepts from object databases leading to object relational.
- Object Database Standard: ODMG-93, ODMG-version 2.0, and ODMG-version 3.0.
- Exemplified in the latest versions of Oracle-10i, DB2, and SQL Server and other DBMSs.
- Standards included in SQL-99 and expected to be enhanced in future SQL standards.

1.3.1.4 Network Model

The data in the network model are represented by collection of records and relationships among data are represented by links, which can be viewed as pointers.



The records in the database are organized as collection of arbitrary groups.

Advantages:

1. Representation of relationship between entities is implemented using pointers which allows the representation of arbitrary relationship
2. Unlike the hierarchical model it is easy.
3. Data manipulation can be done easily with this model.

1.3.1.5 Hierarchical Model

A hierarchical data model is a data model which the data is organized into a tree like structure. The structure allows repeating information using parent/child relationships: each parent can have many children but each child only has one parent(Fig 1.2). All attributes of a specific record are listed under an entity type.

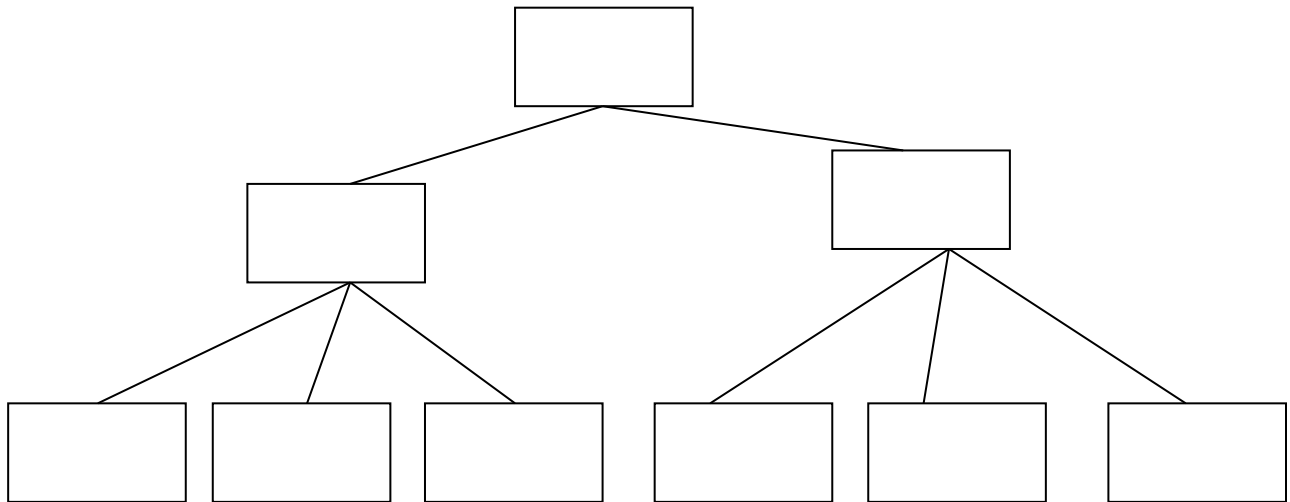


Fig 1.2: Hierarchical Model

Advantages:

1. The representation of records is done using an ordered tree, which is natural method of implementation of one-to-many relationships.
2. Proper ordering of the tree results in easier and faster retrieval of records.
3. Allows the use of virtual records.

1.4 DATABASE SYSTEM ARCHITECTURE

The functional components of a database system can be broadly divided into the storage manager and the query processor components. The storage manager is important because databases typically require a large amount of storage space.

1.4.1 Storage Manager

The storage manager translates the various DML statements into low-level file-system commands. The storage manager is responsible for storing, retrieving, and updating data in the database.

- **Authorization and integrity manager**, which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
- **Transaction manager**, which ensures that the database remains in a consistent (correct) state despite system failures, and that concurrent transaction executions proceed without conflicting.
- **File manager**, which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.

- **Buffer manager**, which is responsible for fetching data from disk storage into main memory, and deciding what data to cache in main memory.

The storage manager implements several data structures as part of the physical system implementation(Fig 1.3a):

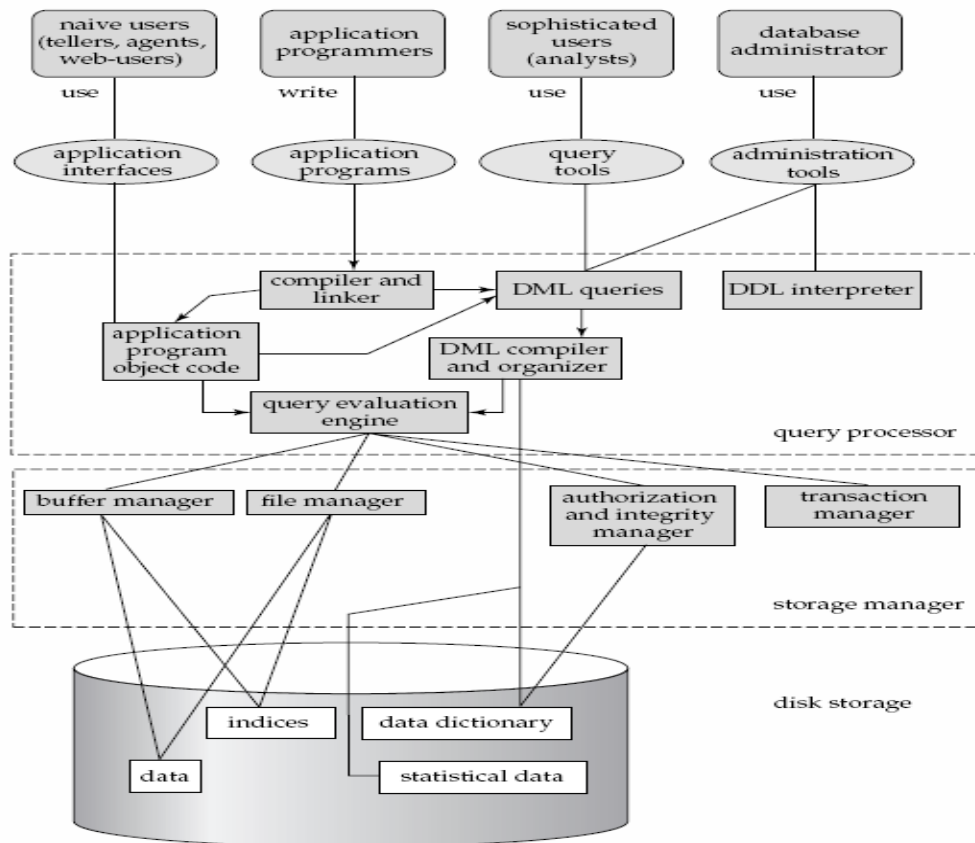


Fig:1.3 a Database System Architecture

- **Data files**, which store the database itself.
- **Data dictionary**, which stores metadata about the structure of the database, in particular the schema of the database.
- **Indices**, which provide fast access to data items that hold particular values.

1.4.2 The Query Processor

The query processor components include

- **DDL interpreter**, which interprets DDL statements and records the definitions in the data dictionary.

- **DML compiler**, which translates DML statements in a query language into an evaluation plan consisting of low-level instructions
- **Query evaluation engine**, which executes low-level instructions

Database Users:

Users are differentiated by the way they expect to interact with the system

- **Application programmers** – interact with the system through DML calls
- **Sophisticated users** – form requests in a database query language
- **Specialized users** – write specialized database applications that do not fit into the traditional data processing framework
- **Naive users** – invoke one of the permanent application programs that have been written previously

Database applications are usually partitioned into two or three parts, as in Fig 1.3.b

- In a **two-tier architecture**, the application resides at the client machine, where it invokes database system functionality at the server machine through query language statements. Application program interface standards like ODBC and JDBC are used for interaction between the client and the server.
- In contrast, in a **three-tier architecture**, the client machine acts as merely a front end and does not contain any direct database calls. Instead, the client end communicates with an **application server**, usually through a forms interface. The application server in turn communicates with a database system to access data.

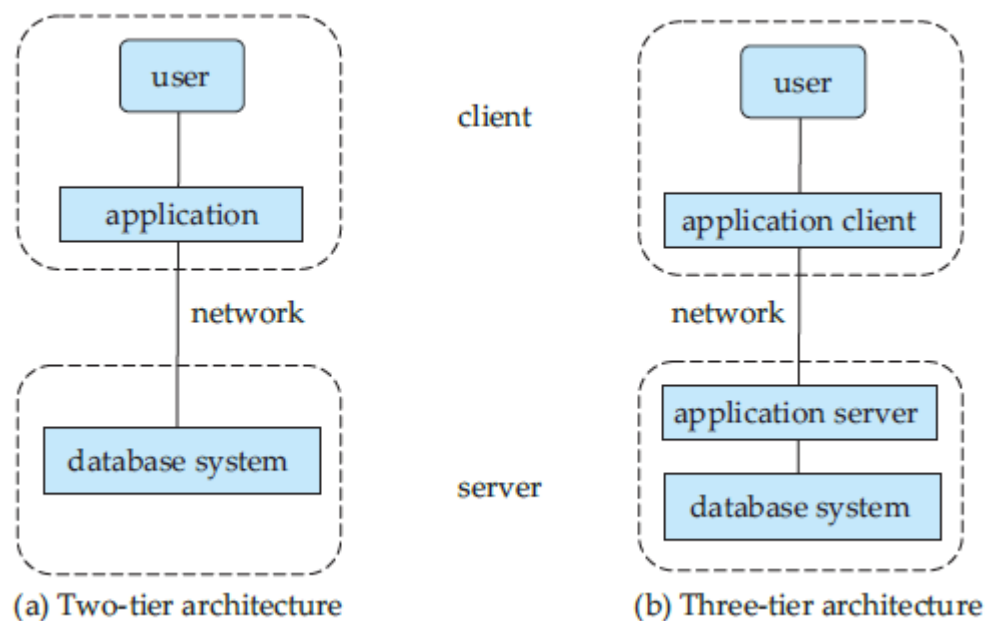


Fig 1.3 b One tier and Two Tier Architectures

1.5 INTRODUCTION TO THE RELATIONAL MODEL

The Relational model is today the primary data model for commercial data processing applications.

1.5.1 Structure of Relational Databases

- A relational database consists of a collection of **tables**, each of which is assigned a unique name. For example, consider the *instructor* table of Fig 1.4, which stores information about instructors.
 - The table has four column headers: *ID*, *name*, *dept name*, and *salary*. Each row of this table records information about an instructor, consisting of the instructor's *ID*, *name*, *dept name*, and *salary*.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Fig 1.4 Instructor Relation

- Thus, in the relational model the term **relation** is used to refer to a table, while the term **tuple** is used to refer to a row. Similarly, the term **attribute** refers to a column of a table.
- For each attribute of a relation, there is a set of permitted values, called the domain of that attribute.
- A domain is atomic if elements of the domain considered to be indivisible.

1.5.2 Database Schema

- The concept of a relation corresponds to the programming-language notion of a variable, while the concept of a **relation schema** corresponds to the programming-language notion of type definition.
- In general, a relation schema consists of a list of attributes and their corresponding domains.
- The concept of a relation instance corresponds to the programming-language notion of a value of a variable.

The value of a given variable may change with time; similarly the contents of a relation instance may change with time as the relation is updated. In contrast, the schema of a relation does not generally change.

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Fig 1.5 The Department Relation

Consider the *department* relation of Figure 1.5. The schema for that relation is *department* (*dept name*, *building*, *budget*) Note that the attribute *dept name* appears in both the *instructor* schema and the *department* schema.

Each course in a university may be offered multiple times, across different semesters, or even within a semester. We need a relation to describe each individual offering, or section, of the class.

The schema is *section* (*course id*, *sec id*, *semester*, *year*, *building*, *room number*, *time slot id*)

1.5.3 Keys

- We must have a way to specify how tuples within a given relation are distinguished. This is expressed in terms of their attributes.

- In other words, no two tuples in a relation are allowed to have exactly the same value for all attributes.
- A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in the relation. For example, the *ID* attribute of the relation *instructor* is sufficient to distinguish one instructor tuple from another. Thus, *ID* is a superkey.
- We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called **candidate keys**.
- A relation, say *r1*, may include among its attributes the primary key of another relation, say *r2*. This attribute is called a **foreign key** from *r1*, referencing *r2*. The relation *r1* is also called the **referencing relation** of the foreign key dependency, and *r2* is called the **referenced relation** of the foreign key.

1.5.4 Schema Diagrams

- A database schema, along with primary key and foreign key dependencies, can be depicted by **schema diagrams**. Figure 1.6 shows the schema diagram for our university organization.
- Each relation appears as a box, with the relation name at the top in blue, and the attributes listed inside the box. Primary key attributes are shown underlined.
- Foreign key dependencies appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation.

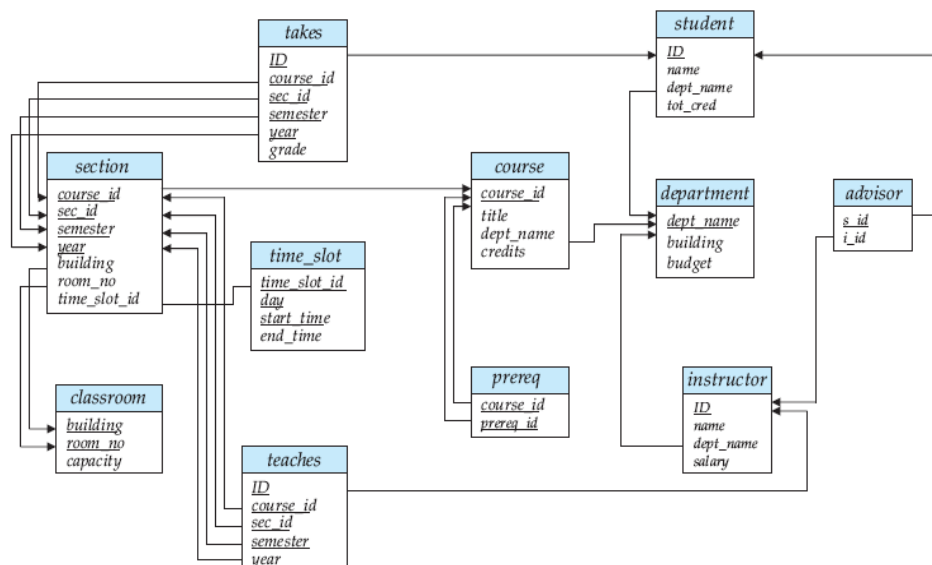


Fig 1.6 Schema diagram for the university database

1.5.5 Relational Query Languages

- A **query language** is a language in which a user requests information from the database. These languages are usually on a level higher than that of a standard programming language.
- Query languages can be categorized as either procedural or nonprocedural.
- In a **procedural language**, the user instructs the system to perform a sequence of operations on the database to compute the desired result.
- In a **nonprocedural language**, the user describes the desired information without giving a specific procedure for obtaining that information.
- There are a number of “pure” query languages: The relational algebra is procedural, whereas the tuple relational calculus and domain relational calculus are nonprocedural.

1.5.6 Relational Operations

- All procedural relational query languages provide a set of operations that can be applied to either a single relation or a pair of relations.
- The *join* operation allows the combining of two relations by merging pairs of tuples, one from each relation, into a single tuple.
- The *Cartesian product* operation combines tuples from two relations, but unlike the join operation, its result contains *all* pairs of tuples from the two relations, regardless of whether their attribute values match.
- The *union* operation performs a set union of two “similarly structured” tables (say a table of all graduate students and a table of all undergraduate students). For example, one can obtain the set of all students in a department. Other set operations, such as *intersection* and *set difference* can be performed as well.

1.6 RELATIONAL ALGEBRA:

Basic operations:

- Selection (σ) Selects a subset of rows from relation.
- Projection (π) Selects a subset of columns from relation.
- Cross-product ($\times$) allows us to combine two relations.
- Set-difference ($-$) Tuples in relation1, but not in relation2.
- Union ($\cup$) Tuples in relation1 and in relation2.
- Rename (ρ) Use new name for the Tables or fields.

Additional operations: Intersection ($\cap$), join ($\bowtie$), division ($\div$): Not essential, but (very!) useful.

Since each operation returns a relation, operations can be composed! (Algebra is “closed”.)

Projection: Deletes attributes that are not in projection list.

Schema of result contains exactly the fields in the projection list, with the same names that they had in the (only) input relation. (Unary Operation)

Projection operator has to eliminate duplicates! (as it returns a relation which is a set)

Note: real systems typically don’t do duplicate elimination unless the user explicitly asks for it. (Duplicate values may be representing different real world entity or relationship)

Consider the BOOK table:

Acc-No	Title	Author
100	“DBMS”	“Silbershatz”
200	“DBMS”	“Ramanuj”
300	“COMPILER”	“Silbershatz”
400	“COMPILER”	“Ullman”
500	“OS”	“Sudarshan”
600	“DBMS”	“Silbershatz”

$\pi_{\text{Title}}(\text{BOOK}) =$

Title
“DBMS”
“COMPILER”
“OS”

Selection:

Selects rows that satisfy selection condition.

No duplicates in result.

Schema of result identical to schema of (only) input relation.

Result relation can be the input for another relational algebra operation! (Operator composition.)

$\sigma_{\text{Acc-no} > 300}(\text{BOOK}) =$

Acc-No	Title	Author
400	“COMPILER”	“Ullman”
500	“OS”	“Sudarshan”
600	“DBMS”	“Silbershatz”

$\sigma_{\text{Title} = \text{“DBMS”}}(\text{BOOK}) =$

$\pi_{\text{Acc-no}}(\sigma_{\text{Title} = \text{“DBMS”}}(\text{BOOK})) =$

Acc-No

Acc-No	Title	Author
100	“DBMS” ”	“Silbershatz”
200	“DBMS” ”	“Ramanuj”
600	“DBMS” ”	“Silbershatz”

100
200
600

Union, Intersection, Set-Difference

All of these operations take two input

relations, which must be union-compatible:

Same number of fields.

‘Corresponding’ fields have the same type.

What is the schema of result?

Consider:

Borrower

Depositor

Cust-name	Loan-no
Ram	L-13
Shyam	L-30
Suleman	L-42

Cust-name	Acc-no
Suleman	A-100
Radheshyam	A-300
Ram	A-401

List of customers who are either borrower or depositor at bank = $\pi_{\text{Cust-name}}(\text{Borrower}) \cup$

$\pi_{\text{Cust-name}}(\text{Depositor}) =$

Cust-name
Ram
Shyam
Suleman
Radeshyam

Customers who are both borrowers and depositors = $\pi_{\text{Cust-name}}(\text{Borrower}) \cap \pi_{\text{Cust-name}}(\text{Depositor}) =$

Cust-name
Ram
Suleman

Customers who are borrowers but not depositors = $\pi_{\text{Cust-name}}(\text{Borrower}) - \pi_{\text{Cust-name}}(\text{Depositor}) =$

Cust-name
Shyam

Cartesian-Product or Cross-Product ($S1 \times R1$)

Each row of S1 is paired with each row of R1.

Result schema has one field per field of S1 and R1, with field names 'inherited' if possible.

Consider the borrower and loan tables as follows:

Borrower:

Cust-name	Loan-no
Ram	L-13
Shyam	L-30
Suleman	L-42

Loan:

Loan-no	Amount
L-13	1000
L-30	20000
L-42	40000

Cross product of Borrower and Loan, $\text{Borrower} \times \text{Loan} =$

Borrower.Cust-name	Borrower.Loan-no	Loan. Loan-no	Loan. Amount
Ram	L-13	L-13	1000
Ram	L-13	L-30	20000
Ram	L-13	L-42	40000
Shyam	L-30	L-13	1000

Shyam	L-30	L-30	20000
Shyam	L-30	L-42	40000
Suleman	L-42	L-13	1000
Suleman	L-42	L-30	20000
Suleman	L-42	L-42	40000

The rename operation can be used to rename the fields to avoid confusion when two field names are same in two participating tables:

For example the statement, $\rho_{\text{Loan-borrower}}(\text{Cust-name}, \text{Loan-No-1}, \text{Loan-No-2}, \text{Amount})(\text{Borrower} \times \text{Loan})$ results into- A new Table named Loan-borrower is created where it has four fields which are renamed as Cust-name, Loan-No-1, Loan-No-2 and Amount and the rows contains the same data as the cross product of Borrower and Loan.

Loan-borrower:

Cust-name	Loan-No-1	Loan-No-2	Amount
Ram	L-13	L-13	1000
Ram	L-13	L-30	20000
Ram	L-13	L-42	40000
Shyam	L-30	L-13	1000
Shyam	L-30	L-30	20000
Shyam	L-30	L-42	40000
Suleman	L-42	L-13	1000
Suleman	L-42	L-30	20000
Suleman	L-42	L-42	40000

Rename Operation:

It can be used in two ways:

$\rho_x(E)$ return the result of expression E in the table named x.

$\rho_{x(A_1, A_2, \dots, A_n)}(E)$ return the result of expression E in the table named x with the attributes renamed to A1, A2, ..., An.

Its benefit can be understood by the solution of the query “Find the largest account balance in the bank”

It can be solved by following steps:

Find out the relation of those balances which are not largest.

Consider Cartesian product of Account with itself i.e. $\text{Account} \times \text{Account}$

Compare the balances of first Account table with balances of second Account table in the product.

For that we should rename one of the account tables by some other name to avoid the confusion

It can be done by following operation

$\Pi_{\text{Account.balance}} (\sigma_{\text{Account.balance} < \text{d.balance}(\text{Account} \times \rho_d(\text{Account}))})$

So the above relation contains the balances which are not largest.

Subtract this relation from the relation containing all the balances i.e. $\Pi_{\text{balance}}(\text{Account})$.

So the final statement for solving above query is

$\Pi_{\text{balance}}(\text{Account}) - \Pi_{\text{Account.balance}} (\sigma_{\text{Account.balance} < \text{d.balance}(\text{Account} \times \rho_d(\text{Account}))})$

Additional Operations

Natural Join ($S_1 \bowtie R_1$)

Forms Cartesian product of its two arguments, performs selection forcing equality on those attributes that appear in both relations

For example consider Borrower and Loan relations, the natural join between them $\text{Borrower} \bowtie \text{Loan}$ will automatically perform the selection on the table returned by $\text{Borrower} \times \text{Loan}$ which force equality on the attribute that appear in both Borrower and Loan i.e. Loan-no and also will have only one of the column named Loan-No.

That means

$\text{Borrower} \bowtie \text{Loan} = \sigma_{\text{Borrower.Loan-no} = \text{Loan.Loan-no}} (\text{Borrower} \times \text{Loan})$.

The table returned from this will be as follows:

Eliminate rows that does not satisfy the selection criteria “ $\sigma_{\text{Borrower.Loan-no} = \text{Loan.Loan-no}}$ ” from $\text{Borrower} \times \text{Loan} =$

Borrower. Cust-name	Borrower. Loan-no	Loan. Loan-no	Loan. Amount
Ram	L-13	L-13	1000
Ram	L-13	L-30	20000

Ram	L-13	L-42	40000
Shyam	L-30	L-13	1000
Shyam	L-30	L-30	20000
Shyam	L-30	L-42	40000
Suleman	L-42	L-13	1000
Suleman	L-42	L-30	20000
Suleman	L-42	L-42	40000

And will remove one of the columns named Loan-no.

i.e. **Borrower** ⋈ **Loan** =

Cust-name	Loan-no	Amount
Ram	L-13	1000
Shyam	L-30	20000
Suleman	L-42	40000

Division Operation:

Denoted by $\div$ is used for queries that include the phrase “for all”.

For example “Find customers who have an account in all branches in branch city Agra”. This query can be solved by following statement.

$\Pi_{\text{Customer-name, branch-name}} (\text{Depositor} \bowtie \text{Account}) \div \Pi_{\text{branch-name}} (\sigma_{\text{Branch-city} = \text{Agra}} (\text{Branch}))$

The division operations can be specified by using only basic operations as follows: Let $r(R)$ and $s(S)$ be given relations for schema R and S with $S \subseteq R$

$$r \div s = \Pi_{R-S}(r) - \Pi_{R-S} ((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

1.7 FUNDAMENTALS OF SQL

SQL is a language to operate databases; it includes database creation, deletion, fetching rows, modifying rows, etc. SQL is an **ANSI** (American National Standards Institute) standard language, but there are many different versions of the SQL language.

Definition:

- SQL is Structured Query Language, which is a computer language for storing, manipulating and retrieving data stored in a relational database.

- SQL is the standard language for Relational Database System. All the Relational Database Management Systems (RDMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their standard database language.

Also, they are using different dialects, such as –

- MS SQL Server using T-SQL,
- Oracle using PL/SQL,
- MS Access version of SQL is called JET SQL (native format) etc.

Use Of SQL

SQL is widely popular because it offers the following advantages –

- Allows users to access data in the relational database management systems.
- Allows users to describe the data.
- Allows users to define the data in a database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create view, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views.

A Brief History of SQL

- **1970** – Dr. Edgar F. "Ted" Codd of IBM is known as the father of relational databases. He described a relational model for databases.
- **1974** – Structured Query Language appeared.
- **1978** – IBM worked to develop Codd's ideas and released a product named System/R.
- **1986** – IBM developed the first prototype of relational database and standardized by ANSI. The first relational database was released by Relational Software which later came to be known as Oracle.

such as Java, COBOL, or C/C++.

Types Of SQL Commands

- Data Definition Language[DDL]
- Data Manipulation Language [DML]
- Data Control Language [DCL]
- Data Query Language [DQL]
- Data Administration Statement[DAS]
- Transaction Control Statement[TCS]

The Data Definition Language (DDL).

This component of the SQL language is used to create and modify tables and other objects in the database. For tables there are three main commands:

- **CREATE TABLE** tablename to create a table in the database
- **DROP TABLE** tablename to remove a table from the database
- **ALTER TABLE** tablename to add or remove columns from a table in the database

The Data Manipulation Language (DML)

DML component of the SQL language is used to manipulate data within a table. There are four main commands:

- **SELECT** to select rows of data from a table
- **INSERT** to insert rows of data into a table
- **UPDATE** to change rows of data in a table
- **DELETE** to remove rows of data from a table

The Data Control Language (DCL)

This component of the SQL language is used to create privileges to allow users access to, and manipulation of, the database. There are two main commands:

- **GRANT** to grant a privilege to a user
- **REVOKE** to revoke (remove) a privilege from a user

Transaction Control Language (TCL)

The commands of SQL that are used to control the transactions made against the database are collectively called as TCL and examples include Commit, Rollback and Savepoint.

- **Commit:** Commit is used for the permanent changes. When we use Commit in any query then the change made by that query will be permanent and visible. We can't Rollback after the Commit.
- **Rollback:** Rollback is used to undo the changes made by any command but only before a commit is done. We can't Rollback data which has been committed in the database with the help of the commit keyword.
- **Save point:** creates points within groups of transactions in which to ROLLBACK.

DQL (Data Query Language)

It is the component of SQL statement that allows getting data from the database. **Example:**
SELECT

1.7.1 Data Definition Language [DDL]

The SQL data-definition language (DDL) is used to define relational database of a system. The general syntax of a system is

VERB(parameter 1, parameter 2,, parameter n);

Create Database Construct:

The SQL CREATE DATABASE statement is used to create a new SQL database.

Syntax

CREATE DATABASE statement is as follows:

CREATE DATABASE DatabaseName;

Always the database name should be unique within the RDBMS.

Create Table Construct:

Creating a basic table involves naming the table and defining its columns and each column's data type. The SQL CREATE TABLE statement is used to create a new table.

Syntax

The basic syntax of the CREATE TABLE statement is as follows:

CREATE TABLE table_name

(column1 datatype, column2 datatype,
column3 datatype, columnN datatype,

PRIMARY KEY(one or more columns));

CREATE TABLE is the keyword telling the database system what you want to do. In this case, you want to create a new table. The unique name or identifier for the table follows the CREATE TABLE statement.

Then in brackets comes the list defining each column in the table and what sort of data type it is.

The syntax becomes clearer with the following example. A copy of an existing table can be created using a combination of the CREATE TABLE statement and the SELECT statement. You can check the complete details at Create Table Using another Table.

SQL> CREATE TABLE CUSTOMERS

(ID INT NOT NULL, NAME VARCHAR (20) NOT NULL,
AGE INT NOT NULL, ADDRESS CHAR (25) ,
SALARY DECIMAL (18, 2), PRIMARY KEY (ID));

You can verify if your table has been created successfully by looking at the message displayed by the SQL server, otherwise you can use the DESC command as follows:

SQL> DESC CUSTOMERS;

Field	Type	Null	Key	Default	Extra
ID	int(11)	NO	PRI		
NAME	varchar(20)	NO			
AGE	int(11)	NO			
ADDRESS	char(25)	YES		NULL	
SALARY	decimal(18,2)	YES		NULL	

5 rows in set (0.00 sec)

Drop and Alter Table Constructs

SQL>drop table *student*

Deletes the table and its contents

SQL>delete from *student*

Deletes all contents of table, but retains table

Alter Table

SQL>alter table *r* add *A D*

- where *A* is the name of the attribute to be added to relation *r* and *D* is the domain of *A*.
- All tuples in the relation are assigned *null* as the value for the new attribute.

SQL>alter table *r* drop *A*

- where *A* is the name of an attribute of relation *r*
- Dropping of attributes not supported by many databases.

1.7.2 Data Manipulation Language

The SQL **data-manipulation language (DML)** provides the ability to query information, and insert, delete and update tuples

A typical SQL query has the form:

```
select A1, A2, ..., An
from r1, r2, ..., rm
where P
```

where, *A_i* represents an attribute, *R_i* represents a relation, *P* is a predicate.

The Select Clause

The **select** clause list the attributes desired in the result of a query and it corresponds to the projection operation of the relational algebra. SQL names are case insensitive

Example: find the names of all instructors

SQL>select name from instructor

SQL allows duplicates in relations as well as in query results. To force the elimination of duplicates, insert the keyword **distinct** after select.

Find the names of all departments with instructor, and remove duplicates

SQL>select distinct dept_name from instructor

The keyword **all** specifies that duplicates not be removed.

SQL>select all dept_name from instructor

An asterisk in the select clause denotes —all attributes

SQL>select * from instructor

The **select** clause can contain arithmetic expressions involving the operation, +, −, *, and /, and operating on constants or attributes of tuples. The query:

SQL>select ID, name, salary/12 from instructor

would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.

The Where Clause

The **where** clause specifies conditions that the result must satisfy and it corresponds to the selection predicate of the relational algebra.

To find all instructors in IT department with salary > 80000

SQL>select name from instructor where dept_name = 'IT.' and salary > 80000

Comparison results can be combined using the logical connectives **and**, **or**, and **not**. It can be applied to results of arithmetic expressions.

The From Clause

The **from** clause lists the relations involved in the query and corresponds to the Cartesian product operation of the relational algebra.

Find the Cartesian product *instructor X teaches*

SQL>select * from instructor, teaches

- It generates every possible instructor – teaches pair, with all attributes from both relations .
- Cartesian product is not very useful directly, but useful when combined with where-clause condition(selection operation in relational algebra).

The three data manipulation statements: INSERT, DELETE, UPDATE

Modification of database:

- Deletion of tuples from a given relation
- Insertion of new tuples into a given relation
- Updating values in some tuples in a given relation

1.7.2.1 Insert Command

INSERT is used to add a single tuple to a relation.

SQL>INSERT INTO EMPLOYEE

VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
Oak Forest, Katy, TX', 'M', 37000, '653298653', 4);

A second form of the INSERT statement allows the user to specify explicit attribute names that correspond to the values provided in the INSERT command. the values must include all attributes with NOT NULL specification *and* no default value. Attributes with NULL allowed or DEFAULT values are the ones that can be *left out*.

SQL>INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)

VALUES ('Richard', 'Marini', 4, '653298653');

A DBMS that fully implements SQL should support and enforce all the integrity constraints that can be specified in the DDL.

A variation of the INSERT command inserts multiple tuples into a relation in conjunction with creating the relation and loading it with the *result of a query*.

SQL>CREATE TABLE WORKS_ON_INFO (Emp_name VARCHAR(15),

Proj_name VARCHAR(15), Hours_per_week DECIMAL(3,1));

SQL>INSERT INTO WORKS_ON_INFO (Emp_name, Proj_name, Hours_per_week)

SQL>SELECT E.Lname, P.Pname, W.Hours

FROM PROJECT P, WORKS_ON W, EMPLOYEE E

WHERE P.Pnumber=W.Pno AND W.Essn=E.Ssn;

A table WORKS_ON_INFO is created and is loaded with the joined information retrieved from the database by the query insert.

1.7.2.2 Delete Command

The DELETE command removes tuples from a relation. It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted. Tuples are explicitly deleted from only one table at a time. Use the DROP TABLE command to remove the table definition.

```
DELETE FROM EMPLOYEE  
WHERE Dno=5;  
DELETE FROM EMPLOYEE;
```

1.7.2.3 Update Command

The **UPDATE** command is used to modify attribute values of one or more selected tuples. As in the DELETE command, a WHERE clause in the UPDATE command selects the tuples to be modified from a single relation. An additional **SET** clause in the UPDATE command specifies the attributes to be modified and their new values.

```
UPDATE PROJECT  
SET Plocation = 'Bellaire', Dnum = 5 WHERE Pnumber=10;
```

1.7.3 Data Control Language (DCL)

It is related to the security issues, that is, determining who has access to database objects and what operations they can perform on them.

The Database Administrator has the power to give and take the privileges to a specific user, thus giving or denying access to the data.

The DCL commands are GRANT and REVOKE

Grant

The **grant** statement is used to confer authorization. The basic form of this statement is:

Syntax:

```
GRANT <object privileges>  
ON <object_name>  
TO <User_Name>  
[WITH GRANT OPTION]
```

eg:

```
GRANT ALL  
ON Student TO Mohd Imran  
WITH GRANT OPTION
```

The WITH GRANT OPTION allows the grantee to in turn grant object privileges to other users.

eg:

```
GRANT SELECT, UPDATE  
ON Student TO Fareen
```

WITH GRANT OPTION

The user Fareen has been given permission to view and modify records in the table Student. To view the contents of the student table that belongs to Mohd Imran.

eg: `SELECT * FROM Mohd Imran.Student;`

Mohd Imran is the owner of the table, and he has given privileges to other user for his table.

Objects Privileges that can be given to users.

Each object privileges that is granted authorized the grantee to perform some operation on the object. A user can grant all the privileges or grant only specific object privileges.

The list of object privileges is as follows:

- **ALTER:** Allows the grantee to change the table definition with the ALTER TABLE command.
- **DELETE:** Allows the grantee to remove the records from the table with the DELETE command.
- **INDEX:** Allows the grantee to create an index on the table with the CREATE INDEX command.
- **INSERT:** Allows the grantee to add records to the table with the INSERT command.
- **SELECT:** Allows the grantee to query the table with the SELECT command.
- **UPDATE:** Allows the grantee to modify the records in the tables with the UPDATE command.

Revoking Privileges Given:

Privileges once given can be denied to a user using the REVOKE command. The object owner can revoke privileges granted to another user. A user of an object who is not the owner, but has been granted the GRANT privileges, has the power to REVOKE the privileges from grantee.

Revoking the Permission using the REVOKE command.

Syntax:

```
REVOKE <Object_Privileges>  
ON <Object_Name>  
FROM <User_Name>
```

Eg:

```
REVOKE UPDATE
```

ON Student
FROM Fareen;

To revoke the privileges that were granted previously, write

- **revoke select on department from Amit, Satoshi;**
- **revoke update (budget) on department from Amith, Satoshi;**

Revocation of privileges is complex if the user from whom the privilege is revoked has granted the privilege to another user.

<privilege-list> may be **all** to revoke all privileges the revokee may hold.

If <revokee-list> includes **public**, all users lose the privilege except those granted it explicitly.

- If the same privilege was granted twice to the same user by different grantees, the user may retain the privilege after the revocation.
- All privileges that depend on the privilege being revoked are also revoked

1.7.4 Transaction Control Language (TCL)

- Transaction Control statements are COMMIT, ROLLBACK and SAVEPOINTS.
- These statements manage all the changes made by the DML statements. For example transaction statements commit data.
- **Commit work** commits the current transaction.
- It makes the updates performed by the transaction become permanent the database.
- After the transaction is committed, a new transaction is automatically started.
- **Rollback work** causes the current transaction to be rolled back.
- It undoes all the updates performed by the SQL statements in the transaction.
- Thus the database state is restored to what it was before the first statement of the transaction was executed.

Savepoints

- A **savepoint (syncpoint or checkpoint)** is a point in the transaction that can be rolled back to without rolling back the entire transaction.
- **Savepoints** can be establish inside the transaction. While the transaction is being executed issue either a COMMIT or ROLLBACK command to commit or discard the changes since a particular savepoint.

Syntax: SAVEPOINT savepoint-name

Example: Consider a transaction that has five delete operations. Now the transaction has been started and deleted the records. After each deletion the savepoints have been created as shown below:

SAVEPOINT SP1

delete from employee where empNo=100;

SAVEPOINT SP2

delete from employee where empNo=110;

SAVEPOINT SP3

delete from employee where empNo=120;

SAVEPOINT SP4

delete from employee where empNo=130;

SAVEPOINT SP5

delete from employee where empNo=140;

1.8 ADVANCED SQL FEATURES

- Accessing SQL from a Programming Language
 - Java Database Connectivity(JDBC)
 - Open Database Connectivity(ODBC)
 - Embedded SQL
- Functions and Procedures
- Triggers
- Recursive Queries
- Advanced Aggregation Features
- Online Analytical Processing(OLAP)

1.8.1 Accessing SQL From a Programming Language

SQL provides a powerful declarative query language. Writing queries in SQL is usually much easier than coding the same queries in a general-purpose programming language.

There are two approaches to accessing SQL from a general-purpose programming language:

- Dynamic SQL

- Embedded SQL

1.8.1.1 JDBC

The **JDBC** standard defines an **application program interface (API)** that Java programs can use to connect to database servers. (The word JDBC was originally an abbreviation for **Java Database Connectivity**, but the full form is no longer used.)

```
public static void JDBCexample(String userid, String passwd)
{ try
{ Class.forName ("oracle.jdbc.driver.OracleDriver");
Connection conn = DriverManager.getConnection(
"jdbc:oracle:thin:@db.yale.edu:1521:univdb",
userid, passwd);
Statement stmt = conn.createStatement();
try { stmt.executeUpdate(
"insert into instructor values('77987', 'Kim', 'Physics', 98000)");
} catch (SQLException sqle)
{ System.out.println("Could not insert tuple. " + sqle);
}ResultSet rset = stmt.executeQuery(
"select dept name, avg (salary) "+
" from instructor "+
" group by dept name");
while (rset.next()) { System.out.println(rset.getString("dept name") + " " +
rset.getFloat(2));
}stmt.close();
conn.close();
}catch (Exception sqle)
{ System.out.println("Exception : " + sqle);
}
}
```

1.8.1.2 ODBC

- The **Open Database Connectivity** (ODBC) standard defines an API that applications can use to open a connection with a database, send queries and updates, and get back results.
- Applications such as graphical user interfaces, statistics packages, and spreadsheets can make use of the same ODBC API to connect to any database server that supports ODBC.
- Each database system supporting ODBC provides a library that must be linked with the client program. When the client program makes an ODBC API call, the code in the library communicates with the server to carry out the requested action, and fetch results.

1.8.2 Functions and Procedures

- Functions are particularly useful with specialized data types such as images and geometric objects. For instance, a line-segment data type used in a map database may have an associated function that checks whether two line segments overlap, and an image data type may have associated functions to compare two images for similarity.
- Procedures and functions allow “business logic” to be stored in the database, and executed from SQL statements. For example, universities usually have many rules about how many courses a student can take in a given semester, the minimum number of courses a full-time instructor must teach in a year, the maximum number of majors a student can be enrolled in, and so on.
- SQL allows the definition of functions, procedures, and methods. These can be defined either by the procedural component of SQL, or by an external programming language such as Java, C, or C++.

1.8.3 Triggers

A **trigger** is a statement that the system executes automatically as a side effect of a modification to the database. To design a trigger mechanism, we must meet two requirements:

1. Specify when a trigger is to be executed. This is broken up into an *event* that causes the trigger to be checked and a *condition* that must be satisfied for trigger execution to proceed.
2. Specify the *actions* to be taken when the trigger executes.

1.8.3.1. Need for Triggers

- Triggers can be used to implement certain integrity constraints that cannot be specified using the constraint mechanism of SQL.
- Triggers are also useful mechanisms for alerting humans or for starting certain tasks automatically when certain conditions are met.

1.8.3.2. Triggers in SQL

Triggers can be disabled or enabled; by default they are enabled when they are created, but can be disabled by using **alter trigger trigger name disable** (some databases use alternative syntax such as **disable trigger trigger name**).

A trigger that has been disabled can be enabled again. A trigger can instead be dropped, which removes it permanently, by using the command **drop trigger trigger name**. Returning to our warehouse inventory example, suppose we have the following relations:

- *inventory (item, level)*, which notes the current amount of the item in the warehouse.
- *minlevel (item, level)*, which notes the minimum amount of the item to be maintained.
- *reorder (item, amount)*, which notes the amount of the item to be ordered when its level falls below the minimum.
- *orders (item, amount)*, which notes the amount of the item to be ordered.

1.8.3.3. When Not to Use Triggers

- There are many good uses for triggers, such as those we have just seen in Section 5.3.2, but some uses are best handled by alternative techniques. For example, we could implement the **on delete cascade** feature of a foreign-key constraint by using a trigger, instead of using the cascade feature.

1.8.4. Recursive Queries

The **transitive closure** of the relation *prereq* is a relation that contains all pairs (*cid*, *pre*) such that *pre* is a direct or indirect prerequisite of *cid*. There are numerous applications that require computation of similar transitive closures on **hierarchies**.

1.8.5. Advanced Aggregation Features

The aggregation support in SQL, which we have seen earlier, is quite powerful, and handles most common tasks with ease. However, there are some tasks that are hard to implement efficiently with the basic aggregation features. In this section, we study features that were added to SQL to handle some such tasks.

1.8.5.1. Ranking

Finding the position of a value in a larger set is a common operation. For instance we may wish to assign students a rank in class based on their grade-point average (GPA), with the rank 1 going to the student with the highest GPA, the rank 2 to the student with the next highest GPA, and so on.

Ranking is done with an **order by** specification. The following query gives the rank of each student:

```
select ID, rank() over (order by (GPA) desc) as s rank  
from student grades;
```

Note that the order of tuples in the output is not defined, so they may not be sorted by rank. An extra **order by** clause is needed to get them in sorted order, as shown below.

```
select ID, rank () over (order by (GPA) desc) as s rank  
from student grades  
order by s rank;
```

1.8.5.2. Windowing

Window queries compute an aggregate function over ranges of tuples. This is useful, for example, to compute an aggregate of a fixed range of time; the time range is called a *window*.

1.8.6 OLAP

- An online analytical processing (OLAP) system is an interactive system that permits an analyst to view different summaries of multidimensional data.
- The word *online* indicates that an analyst must be able to request new summaries and get responses online, within a few seconds, and should not be forced to wait for a long time to see the result of a query.
- There are many OLAP products available, including some that ship with database products such as Microsoft SQL Server, and Oracle, and other standalone tools.
- The initial versions of many OLAP tools assumed that data is memory resident. Data analysis on small amounts of data can in fact be performed using spreadsheet applications, such as Excel.
- However, OLAP on very large amounts of data requires that data be resident in a database, and requires support from the database for efficient preprocessing of data as well as for online query processing

1.9 EMBEDDED SQL

An overview of the technique for how SQL statements can be embedded in a general-purpose programming language. We focus on two languages: C and Java. In this embedded approach, the programming language is called the host language. Most SQL statements—including data or constraint definitions, queries, updates, or view definitions—can be embedded in a host language program.

1.9.1 Retrieving Single Tuples With Embedded SQL

When using C as the host language, an embedded SQL statement is distinguished from programming language statements by prefixing it with the keywords EXEC SQL so that a preprocessor (or precompiler) can separate embedded SQL statements from the host language code.

The SQL statements within a program are terminated by a matching END-EXEC or by a semicolon (;). Shared variables are prefixed by a colon (:) *when they appear in an SQL statement*. This distinguishes program variable names from the names of database schema constructs such as attributes (column names) and relations (table names).

The SQL types INTEGER, SMALLINT, REAL, and DOUBLE are mapped to the C types long, short, float, and double, respectively.

- 0) int loop ;
- 1) EXEC SQL BEGIN DECLARE SECTION ;
- 2) varchar dname [16], fname [16], lname [16], address [31] ;
- 3) char ssn [10], bdate [11], sex [2], minit [2] ;
- 4) float salary, raise ;
- 5) int dno, dnumber ;
- 6) int SQLCODE ; char SQLSTATE [6] ;
- 7) EXEC SQL END DECLARE SECTION ;

Lines 1 and 7, which tell the precompiler to take note of the C variable names between BEGIN DECLARE and END DECLARE because they can be included in embedded SQL statements— as long as they are preceded by a colon (:). Lines 2 through 5 are regular C program declarations.

Connecting to the Database

- The SQL command for establishing a connection to a database has the following form:
 - CONNECT TO <server name>AS <connection name>
 - AUTHORIZATION <user account name and password> ;

The programmer or user can use the <connection name> to change from the currently active connection to a different one by using the following command:

- SET CONNECTION <connection name> ;

Once a connection is no longer needed, it can be terminated by the following command:

- DISCONNECT <connection name> ;

Communicating between the Program and the DBMS Using SQLCODE and SQLSTATE.

The two special communication variables that are used by the DBMS to communicate exception or error conditions to the program are SQLCODE and SQLSTATE. After each database command is executed, the DBMS returns a value in SQLCODE. A value of 0 indicates that the statement was executed successfully by the DBMS.

If SQLCODE > 0 (or, more specifically, if SQLCODE = 100), this indicates that no more data (records) are available in a query result.

If SQLCODE < 0, this indicates some error has occurred.

The definition of SQLCA must be included in the C program by including the following line:
EXEC SQL include SQLCA ;

SQLSTATE was added, which is a string of five characters. A value of '00000' in SQLSTATE indicates no error or exception; other values indicate various errors or exceptions. For example, '02000' indicates 'no more data' when using SQLSTATE.

1.9.2 Retrieving Multiple Tuples With Embedded Sql

Using Cursors

The cursor is declared when the SQL query command is declared in the program. OPEN CURSOR command fetches the query result from the database and sets the cursor to a position *before the first row* in the result of the query. This becomes the current row for the cursor.

Subsequently, FETCH commands are issued in the program; each FETCH moves the cursor to the *next row* in the result of the query, making it the current row and copying its attribute values into the C (host language) program variables specified in the FETCH command by an INTO clause.

The cursor variable is basically an iterator that iterates (loops) over the tuples in the query result—one tuple at a time. A CLOSE CURSOR command is issued to indicate that we are done with processing the result of the query associated with that cursor.

The cursor around the tuples in the query result with greater flexibility, providing random access by position or access in reverse order. When SCROLL is specified on the cursor, the general form of a FETCH command is as follows, with the parts in square brackets being optional:

FETCH [[<fetch orientation>] FROM] <cursor name> INTO <fetch target list> ;

The ORDER BY clause orders the tuples so that the FETCH command will fetch them in the specified order. It is specified in a similar manner to the corresponding clause for SQL queries

1.10 DYNAMIC SQL:

- A general-purpose program can connect to and communicate with a database server using a collection of functions (for procedural languages) or methods (for object-oriented languages).
- Dynamic SQL allows the program to construct an SQL query as a character string at runtime, submit the query, and then retrieve the result into program variables a tuple at a time.
- The *dynamic SQL* component of SQL allows programs to construct and submit SQL queries at runtime.
- we look at two standards for connecting to an SQL database and performing queries and updates.
- One, JDBC, is an application program interface for the Java language. The other, ODBC is an application program interface originally developed for the C language, and subsequently extended to other languages such as C++, C#, and Visual Basic.

SQL defines standards for embedding dynamic SQL calls in a host language, such as C, as in following example:

```
char * sqlprog = "update account
    set balance = balance * 1.05
    where acc_no = ?"
EXEC SQL prepare dynprog from : sqlprog;
char acc[10] = "101" ;
EXEC SQL execute dynprog using : acc;
```

The dynamic SQL program contains acc_no = ?, which is a place holder for a value that is provided when the SQL program is executed.

1.10.1 Difference Between Static (Embedded) SQL and Dynamic (Interactive) SQL

S.No.	Static (embedded) SQL	Dynamic (interactive) SQL
1.	In static SQL how database will be accessed is predetermined in the embedded SQL statement.	In dynamic SQL, how database will be accessed is determined at run time.
2.	It is more swift and efficient.	It is less swift and efficient.
3.	SQL statements are compiled at compile time.	SQL statements are compiled at run time.
4.	Parsing, validation, optimization, and generation of application plan are done at compile time.	Parsing, validation, optimization, and generation of application plan are done at run time.
5.	It is generally used for situations where data is distributed uniformly.	It is generally used for situations where data is distributed non-uniformly.
6.	EXECUTE IMMEDIATE, EXECUTE and PREPARE statements are not used.	EXECUTE IMMEDIATE, EXECUTE and PREPARE statements are used.
7.	It is less flexible.	It is more flexible.

PART-A

Two Marks Questions with Answers:

1. . Differentiate File processing system with Database Management system.
(Nov/Dec'16)

Database Systems	Conventional File systems
Data redundancy is less	Data redundancy is more
Security is high	Security is very low

Database systems are used when security constraints are high	Conventional file systems are used where there is less demand for security constraints.
Database systems define the data in a structured manner. Also there is well defined co-relation among the data.	File systems define the data in un-structured manner. Data is usually in isolated form.
Data inconsistency is less in database systems.	Data inconsistency is more in database systems.

2. What is a weak entity? Give example(Nov/Dec'16)

A **weak entity** is the **entity** which can't be fully identified by its own attributes and takes the foreign key as an attribute (generally it takes the primary key of the **entity** it is related to) in conjunction. **Examples:** The existence of rooms is entirely depended on the existence of a hotel.

3. What is an entity relationship model? (Apr/May'16)

The entity relationship model is a collection of basic objects called entities and relationship among those objects. An entity is a thing or object in the real world that is distinguishable from other objects

4. What are the disadvantages of file processing system? (Apr/May'16)

The disadvantages of file processing systems are

- a) Data redundancy and inconsistency
- b) Difficulty in accessing data
- c) Data isolation
- d) Integrity problems
- e) Atomicity problems
- f) Concurrent access anomalies

5. What is the purpose of database management system(nov/dec'14)

- Problems with File Processing System:
- Data redundancy and inconsistency
- Difficulty in accessing data
- Difficulty in data isolation
- Integrity problems

- Atomicity problems
- Concurrent-access anomalies
- Security problems
- We can solve the above problems using Database System.

6. What is a data model? List the types of data model used?

A database model is the theoretical foundation of a database and fundamentally determines in which manner data can be stored, organized, and manipulated in a database system. It thereby defines the infrastructure offered by a particular database system. The most popular example of a database model is the relational model. Types of data model used

- Hierarchical model
- Network model
- Relational model
- Entity-relationship
- Object-relational model
- Object model

7. What is data base management system?

- A database management system (DBMS) is a software package with computer programs that control the creation, maintenance, and the use of a database.
- It allows organizations to conveniently develop databases for various applications by database administrators (DBAs) and other specialists.
- A database is an integrated collection of data records, files, and other database objects.
- A DBMS allows different user application programs to concurrently access the same database. DBMSs may use a variety of database models, such as the relational model or object model, to conveniently describe and support applications.
- It typically supports query languages, which are in fact high-level programming languages, dedicated database languages that considerably simplify writing database application programs.
- Database languages also simplify the database organization as well as retrieving and presenting information from it.
- A DBMS provides facilities for controlling data access, enforcing data integrity, managing concurrency control, recovering the database after failures and restoring it from backup files, as well as maintaining database security.

8. What are the advantages of using a DBMS?

The advantages of using a DBMS are

- a) Controlling redundancy
- b) Restricting unauthorized access
- c) Providing multiple user interfaces
- d) Enforcing integrity constraints.
- e) Providing backup and recovery

9. Give the levels of data abstraction?

- a) Physical level
- b) Logical level
- c) View level

10. Define instance and schema?

Instance: Collection of data stored in the data base at a particular moment is called an Instance of the database.

Schema: The overall design of the data base is called the data base schema.

11. Define the terms Physical schema, logical schema. ?

Physical schema: The physical schema describes the database design at the physical level, which is the lowest level of abstraction describing how the data are actually stored.

Logical schema: The logical schema describes the database design at the logical level, which describes what data are stored in the database and what relationship exists among the data.

12. What is storage manager?

A storage manager is a program module that provides the interface between the low level data stored in a database and the application programs and queries submitted to the system.

13. What are the components of storage manager?

The storage manager components include

- a) Authorization and integrity manager
- b) Transaction manager
- c) File manager
- d) Buffer manager

14. What is the purpose of storage manager?

The storage manager is responsible for the following

- a) Interaction with the file manager
- b) Translation of DML commands in to low level file system commands

c) Storing, retrieving and updating data in the database

15. What is embedded SQL? What are its advantages? April/May-2011

Embedded SQL is a method of combining the computing power of a programming language and the database manipulation capabilities of SQL. Embedded SQL statements are SQL statements written in line with the program source code of the host language. The embedded SQL statements are parsed by an embedded SQL pre processor and replaced by host language calls to a code library. The output from the pre processor is then compiled by the host compiler. This allows programmers to embed SQL statements in programs written in any number of languages such as: C/C++, COBOL and Fortran.

16. What are the categories of SQL command?

SQL commands are divided in to the following categories:

1. data - definition language
2. data manipulation language
3. Data Query language
4. data control language
5. data administration statements
6. transaction control statements

17. What are the Various types of Keys (Apr/May'17)

- Super Key
- Candidate Key
- Foreign Key

18.What is DBMS. What are the Disadvantages of using DBMS (Apr/May'17)

A **database-management system (DBMS)** is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the **database**, contains information relevant to an enterprise.

Disadvantages Of DBMS:

- Confidentiality, Privacy and Security
- Enterprise Vulnerability
- The cost of using a DBMS

19. What is data definition language? Give example.(Dec 16, April/May 2018)

- Data Definition Language(DDL) is a specialized language used to specify a database schema by a set of definitions.

- It is a language which is used for creating and modifying the structure of tables, views, indexes and so on.

Some of the common commands used in DDL are CREATE, ALTER, DROP

20. Define foreign key. (April/May 2018)

Foreign key is a single attribute or collections of attributes in one table that refers to the primary key of other table.

For example-Consider a student database as Student (RollNo, Name, Address) and Course (CourseId, CourseName, RollNo). Here RollNo is a foreign key.

21. What are the four main characteristics that differentiate the database approach from the file-processing approach? (Nov/Dec 2019)

- Self-describing nature of a database system
- Insulation between programs and data, and data abstraction
- Support of multiple views of the data
- Sharing of data and multiuser transaction processing

22. Express in relational algebra, the division operation (/) using the project, Cartesian product and minus operations. Give a simple example. (Nov/Dec 2019)

Relational algebra is a widely used procedural query language. It collects instances of relations as input and gives occurrences of relations as output. It uses various operation to perform this action.

Example:

$\Pi_{\text{CustomerName, Status}}(\text{Customers})$

A-B

$\sigma_{\text{column 2} = '1'}(A \times B)$

In relational algebra the query will be expressed as follows (the example database "kursdatabas" is found on the home page of this course. *matr* = student number, *namn* = name, *kurskod* = course code, *kön* = sex, *kursanmälan* = course registration):

$\Pi_{\text{matr, namn, kurskod}}(\sigma_{\text{kön} = 'K'}(\text{student}) \times \text{kursanmälan}) \div \Pi_{\text{kurskod}}(\sigma_{\text{matr} = 40101}(\text{kursanmälan}))$

PART-B

16 Marks Questions:

1. Describe the Static and Dynamic SQL in detail. (Apr/May 2019) (13)
2. Explain about different kinds of data models with suitable examples. (Apr/May 2019)

(Apr/May '17)(13)

3. Explain aggregate functions in SQL with example. **(May 2018) (13)**
4. Write DDL,DML,DCL commands for the student database. **(Nov/Dec 2017)**
5. Explain about different DML operations. **(Apr/May '17)**
6. Explain select, project and Cartesian product operations in relational algebra with an example. (6) **(Nov/Dec'16)**
7. Briefly explain about Database system architecture? **(Apr/May'16) (Nov/Dec'15)**
8. Briefly explain about Views of data? **(Apr/May'16)**
9. Explain (i) Data model and its types
(ii) E-R diagram for banking system.
10. Explain about the Purpose of Database system.
11. Explain: Data models, Schemas, and Instances.
12. Explain about Embedded SQL.
13. Explain about Dynamic SQL.
14. i. Consider the following schema:

Suppliers(sid: integer,sname:string,address:string)

Parts(pid:integer,pname:string,colr:string)

Catalog(sid:integer,pid:integer,cost:real)

The key fields are underlined and the domain of each field is listed after the field name. Therefore sid is the key for Suppliers, pid is the key for Parts and sid and pid together form the key Catalog. The Catalog relation lists the prices charged for parts by suppliers.

Write the following queries in relational algebra:

- 1) Find the sids of suppliers who supply some red or green part.
- 2) Find the sids of suppliers who supply some red part or are at 221 packer street.
- 3) Find the pids of parts supplied by atleast two different suppliers. **(6)**
- ii. Sketch the typical component modules of DBMS. Indicate and explain the interaction between those modules of the system. **(Nov/Dec 2019)(7)**

15. i. Consider the schema given in question 14. i. and write the following queries in SQL.

- 1) Find the names of suppliers who supply some red part.
- 2) Find the sids of suppliers who supply some red part and some green part.
- 3) Find the sids of suppliers who supply every red part.
- 4) Find the pids of parts supplied by at least two different suppliers. **(8)**
- ii. Explain the three schema architecture with a neat diagram. **(Nov/Dec 2019)(5)**

